PROJECT DOCUMENT

AutoDev Architect — Frontend Proposal

A complete description of the interactive prototype: design system, navigation architecture, screens, functions, and mapping to the AutoDev platform.

File: `AutoDev Redesign.dc.html` · Single-page React prototype · Light/dark themes · No external dependencies

1. Overview

AutoDev Architect is an open-source platform for **planning, analyzing, patching, validating, and evolving** software projects with GenAI agents, positioned to compete with Codex/Cursor-style code assistants while prioritizing open architecture, self-hosting, *patch-based* changes, and human approval workflows.

This prototype is a **proposed frontend redesign**. The previous frontend (dark navy theme, no component library) suffered from low contrast and unclear hierarchy. The proposal rebuilds the experience as an **"execution control center"** that is editorial and calm — inspired by NotebookLM's serenity — with high contrast, intuitive navigation, and generous whitespace, while preserving the technical density a developer expects.

The prototype is **interactive**: the chat processes messages with simulated streaming, plans and patches are editable, and the flow builder lets you assemble agent pipelines visually. All state is managed client-side, with no real backend.

2. Redesign goals

- **Contrast and legibility** — ink on warm paper (or warm white on charcoal), with color tokens calibrated for both themes.
- **Obvious navigation** — labeled sidebar rail with a clear active state, a contextual top bar, and explanatory tooltips.
- **Editorial, calm aesthetic** — serif typography for headings, ample whitespace, consistent visual rhythm.
- **Balanced density** — enough information per screen without overload; avoid "data slop".

- **Product fidelity** — every screen maps to a real AutoDev subsystem (plans, patches, flows, agents, skills, plugins, MCP).

3. Design system

3.1 Color

Two complete themes, switchable at runtime. Colors are defined as custom properties per `data-theme`, so re-theming is instant. The single accent is an **iris/indigo**; semantic colors (execution green, approval amber, error red) share low saturation so they never compete with the accent.

Token	Light (paper)	Dark (charcoal)	Use
`--bg`	#faf8f4	#100f12	Page background
`--bg-2`	#ffffff	#17151a	Cards, panels
`--fg` / `--fg-2`	#1b1a17 / #57544c	#f4f2ee / #b3afa6	Primary / secondary text
`--line`	#e7e2d8	#2a2830	Borders and dividers
`--accent`	#5a4fe0	#8e88ff	Accent, actions, focus
`--success / --warn / --danger`	green / amber / red	same (lighter)	Semantic states

3.2 Typography

Family	Role	Applied to
Newsreader (serif)	Editorial display	Screen titles, chat empty state, section headings
Instrument Sans	UI	All interface text, buttons, labels
JetBrains Mono	Monospace	Code, diffs, console, IDs, technical metadata

3.3 Shape, spacing, and elevation

Generous radii (9–16 px), soft two-layer shadows (`--shadow-sm` / `--shadow`), ample spacing, and content columns with a max width (~720 px) for comfortable reading. Recurring components: **cards**, status **pills**, **chips**, primary/secondary **buttons**, and **tooltips**.

---

4. Structure and navigation (shell)

The app uses a three-zone persistent layout:

Navigation rail (left, 250 px)

- **Brand** AutoDev · Architect with the logo (rotated square).
- **Workspace switcher** — `acpguedes/autodev · main`.
- **Navigation** — Chat, Plans, Patches, Flows, Sessions, Configuration, Extensions. The active item gets an accent-tinted background and a side bar. Plans and Patches show count *badges*.
- **Provider card** — shows the active provider/model and status; it is **clickable** and opens Configuration.
- **Theme toggle** — light/dark.

Top bar

Contextual title and subtitle per screen, a repository chip (`acpguedes/autodev · main`), a **New session** button, and — on Chat — an **Execution** button that opens/hides the execution panel.

Tooltips

Any clickable element shows an explanatory box after ~1 s of hover. Tooltips **respect the window edges**: they flip upward when the element is near the bottom and are clamped horizontally so they never leave the screen.

5. Screens and functions

5.1 Chat — Execution control center

The main screen. An editorial conversation column (max width, centered) with an **empty state** — a serif headline “What are we building today?” and clickable task suggestions. When you send an instruction:

- The user message is appended and the reply **streams in**: Planner → Coder → Validator agents respond in sequence, with realistic delay.
- The **execution panel** (right) shows a live timeline: planning → repository analysis → patch generation → validation (tests + lint), each step with a status icon, monospace output, and a *spinner* while running.
- A **plan preview** appears in the conversation, with a shortcut to the Plans screen.

The **composer** has a textarea, a provider chip, a context button (`@`), and send (Enter sends, Shift+Enter for a newline). The execution panel is dismissible and reopenable from the top bar.

5.2 Plans — steps with approval gates

A list of execution steps, each with an **approval gate**. Stat cards at the top (step count, approved, file impact). Functions:

- **Inline editing** — each step's title and description are editable directly.
- **Approve / Reject** per step; rejecting removes it from execution (dims the card).
- **Add step** (dashed button) and **remove** (×).
- **Execute approved plan** — marks approved steps as done.

5.3 Patches — diff review

A file panel on the left (with `+/-` counts per file) and a viewer on the right. Two modes:

- **Diff** — a unified diff with line numbers and green/red highlighting of additions/removals, marked `dry-run` (nothing is written without approval).
- **Edit** — the file's resulting content opens in a monospace text editor; **manual edits go into the applied patch**.

Actions: **Apply approved patch** and **Discard**.

5.4 Flows — visual pipeline builder

A graph editor for assembling agent pipelines. Three-column structure:

- **Palette (left)** — three sections: existing *Flows* (a loadable library plus a blank "New"), *Agents* (Planner, Navigator, Analyzer, Coder, Validator, Reviewer, Docs, Security), and *Flow control*.
- **Canvas (center)** — positionable nodes over a dotted grid, curved edges with arrowheads and branch labels (e.g. *yes/no*).
- **Inspector (right)** — editing of the selected node (label + type-specific fields) and deletion.

Flow-control blocks

- **Conditional** — branches on an expression (yes/no branches).
- **Loop** — repeats a segment until a condition or a maximum number of iterations.
- **Human approval** — pauses the flow, waiting for a person.
- **Parallel** — runs simultaneous branches and joins the results.
- **Prompt / Step** — an **intermediate step** where you write a prompt for the next agent, with an **"include previous step output"** option (shown when a previous step exists).
- **Start / End** — entry and exit points.

Canvas interactions

- **Add** — clicking a palette item inserts the node, already connected to the selected node.
- **Drag** — reposition any node.
- **Connect via dots** — selecting a node highlights **dots on the four edges** (top/bottom/left/right) of the others; clicking one creates the relationship, and the edge attaches to that side.
- **Save** — exports the graph as `flow.yaml` (confirmed by a *toast*); **Clear** empties the canvas.

```
flow: review-with-patch
nodes:
  - { id: planner,  agent: planner }
  - { id: cond,     type: condition, expr: "analysis.needs_change == true" }
  - { id: coder,    agent: coder }
  - { id: loop,     type: loop, max: 3 }
  - { id: validate, agent: validator }
edges:
  - { from: cond, to: coder, label: "yes" }
  - { from: cond, to: end,   label: "no" }
```

Figure 1 — Conceptual representation of the default flow assembled in the builder.

5.5 Sessions

A reproducible history of executions. Each session shows its goal, ID, run count, time, and status (running / done / failed) and opens the corresponding chat.

5.6 Configuration

Selectable model provider (`stub` offline, `ollama`, `openai`), model, base URL, project directory, and default goal. Conceptually persisted in `autodev.config.json`.

5.7 Extensions

A hub for extending the platform, with **Agents · Skills · Plugins · MCP** tabs (each with a count). Cards list the item, manifest/version, description, and status. Functions:

- **Enable / Disable** each extension.
- **Create / Install / Add** — opens a modal with a per-type form.
- **Clicking a card opens it for editing** (prefilled modal, "Save changes").
- Agents include an **"Agent instruction (system prompt)"** field, alongside model and allowed tools.

```
name: reviewer
version: 0.1.0
model: qwen2.5-coder:14b
tools: [ fs.read, exec ]
instruction: |
  You are a careful engineer. Produce the smallest
  possible diff that solves the task, preserving the
  repository's style.
```

Figure 2 — Conceptual `agent.yaml` generated when creating/editing an agent.

---

6. Interactions and microinteractions

- **Streaming** chat with a timed sequence of agents and a live execution timeline.
- **Animations** that are subtle: *fade/rise* on messages and cards, *spinner* and pulse on active steps.
- **Tooltips** with a 1 s delay and edge-aware repositioning.

- **Dragging** of nodes and **dot-based connection** in the flow builder.
- **Modals** with an overlay, closing on outside click and on navigation.
- **Inline editing** in plans, patches, and flow nodes.

7. Configurable parameters

Parameter	Values	Effect
`startTheme`	light · dark	Initial theme
`startView`	chat · plans · patches · flows · sessions · config	Initial screen
`seedConversation`	true · false	Start with a sample conversation and execution

8. Mapping to the AutoDev platform

Each screen reflects a real subsystem described in the repository documentation:

Screen / function	Backend concept
Chat + agent streaming	Multi-agent orchestrator (Planner/Navigator/Analyzer/Coder/Validator)
Plans + approval gates	*Plan store* with approve/reject; human-in-the-loop
Patches (dry-run + edit)	Unified-diff engine, applied dry-run by default
Flows (graph + control)	Flow engine (E-series): `flow.yaml`, conditionals, loops, checkpoints, human approval
Extensions · Agents	Versioned `agent.yaml`, Agent Registry, budgets
Extensions · Skills	Discover/invoke skill registry
Extensions · Plugins	`plugin.yaml`, Plugin Host, default-deny permissions
Extensions · MCP	Model Context Protocol client/server (stdio/http)
Configuration	Provider stub/ollama/openai, `autodev.config.json`

9. Technical notes

- **Single page** — one React component with state-driven screen switching; no router.
- **Inline styles** and theme tokens via `data-theme`, painting immediately and re-theming without a reload.
- **No external dependencies** beyond Google Fonts; exportable as **self-contained HTML** that works offline.
- **Typography**: Newsreader, Instrument Sans, JetBrains Mono.

10. Limitations and next steps

- The prototype is an **interactive mockup**: streaming and validation results are simulated; there are no calls to a real backend or to models.
- Persistence is client-memory only (reloading resets the state).

- **Suggested next steps**: wire the screens to the real AutoDev APIs (chat/plans/patches/validation), token-by-token streaming of the agent's reply, real file mentions (`@context`) in the composer, and actual export/execution of `flow.yaml`.